

# I-WebGenBench : Evaluating Interactivity in LLM-Generated Scientific Web Applications

Dasen Dai<sup>1\*</sup>, Biao Wu<sup>2\*</sup>, Meng Fang<sup>3</sup>, Shuoqi Li<sup>1</sup>, Wenhao Wang<sup>1†</sup>

<sup>1</sup>Vast Intelligence Lab, <sup>2</sup>UTS, <sup>3</sup>University of Liverpool

\*Equal contribution, †Corresponding author

## Abstract

Interactive scientific concepts such as optimization dynamics, chemical processes, and economic equilibria are inherently dynamic and require user-driven exploration for effective understanding. Recent advances in LLMs have made it possible to generate complex web applications from natural language, raising the prospect of automatically creating interactive scientific demonstrations from research papers. However, existing evaluation frameworks for LLM-based code generation focus primarily on static correctness or visual fidelity, and fail to assess whether generated interfaces exhibit meaningful, event-driven interactivity. We introduce I-WebGenBench, the first benchmark for evaluating the ability of LLMs to generate interactive scientific web applications from paper-derived specifications. The benchmark consists of 201 curated tasks spanning five scientific domains, each requiring models to construct a complete application from scratch while preserving both scientific fidelity and interaction requirements. We propose a deterministic Interaction Probe that executes semantic user actions and detects DOM mutations as a signal of responsiveness, enabling a dual-metric evaluation with Build Success Rate and Interaction Rate that explicitly decouples compilability from interactivity. Experiments on a diverse set of models reveal a substantial gap between compilation and interaction: while most systems achieve high BSR, their IR remains significantly lower, indicating limited ability to implement functional, event-driven behavior. Further analysis shows that this gap is primarily driven by deficiencies in interaction logic rather than rendering quality, and is strongly correlated with structural properties such as state management complexity and layout design. These findings highlight a fundamental limitation of current LLMs in translating structural generation into interactive behavior, and establish I-WebGenBench as a new testbed for advancing interactive code generation.

## 1 Introduction

Scientific concepts such as gradient descent, chemical kinetics, and Nash equilibria are inherently dynamic and parameter-sensitive. However, they are typically presented in static formats such as PDFs, which limits opportunities for interactive exploration [9, 13, 27, 7, 47, 42]. Recent advances in large language models (LLMs) have significantly improved their ability to generate code, and when combined with agent frameworks, they can now produce increasingly complex applications from natural language descriptions [49, 51, 18, 19, 47, 30, 14, 45, 26, 37]. This progress makes it plausible to automatically generate interactive scientific demonstrations that enable real-time, hands-on understanding of abstract concepts.

However, existing evaluation frameworks for LLM-based code generation do not adequately capture interactive capabilities. Code benchmarks such as HumanEval [6] and SWE-bench [46, 21, 2] focus on algorithmic correctness of isolated programs, ignoring whether generated interfaces respond to user

Table 1: Comparison of I-WebGen-Bench with other repository-level software engineering benchmarks. We follow the statistical protocol of [27] for consistency with prior work.

| Benchmark                  | From Scratch | Number of Files | Lines of Code | Interactive |
|----------------------------|--------------|-----------------|---------------|-------------|
| SWE-Bench [21]             | ×            | 1.7             | 32.8          | ×           |
| SWE-Bench Multimodal* [46] | ×            | 2               | 27            | ×           |
| SWE-Lancer [28]            | ×            | 2               | 55            | ×           |
| WebGenBench [27]           | ✓            | 8.1             | 315.3         | ×           |
| PaperVoyager [13]          | ✓            | 11.9            | 698           | ×           |
| I-WebGenBench              | ✓            | 21.2            | 951           | ✓           |

inputs. Front-end design-to-code benchmarks, including Design2Code [31], Interaction2Code [38], and WebVR [4], primarily assess static visual reconstruction without systematically evaluating interactive behavior. More recent systems such as PaperVoyager [13] attempt to convert research papers into interactive applications, but are limited in scale and do not distinguish between compilability and genuine interactivity. Similarly, WebCompass [22] incorporates interaction evaluation via browser agents, yet lacks a deterministic, DOM-mutation-based metric to decouple execution from responsiveness, and does not analyze structural factors such as state management and layout design. As a result, there is currently no systematic and reliable method to evaluate whether generated scientific web applications exhibit meaningful, event-driven interactivity across diverse domains.

To this end, we introduce I-WebGenBench, a benchmark designed to systematically evaluate the ability of LLM-based agents to generate interactive scientific web applications from paper-derived specifications. Unlike prior code-generation benchmarks that focus on functional correctness or patching existing codebases, our benchmark requires models to construct complete, interactive applications from scratch, capturing both scientific fidelity and event-driven behavior [25, 24, 51, 43, 46, 16]. Two key challenges arise in building such a benchmark: (1) how to curate diverse and scientifically grounded specifications that reflect real-world interactive concepts, and (2) how to reliably evaluate whether generated applications exhibit genuine interactivity beyond successful compilation. I-WebGenBench addresses these challenges through a structured data construction process and a deterministic evaluation pipeline.

To construct the dataset, we first collect a diverse set of research papers across five domains—Biology, Chemistry, Computer Science, Economics, and Human-Computer Interaction (HCI). For each paper, domain experts identify core mechanisms suitable for interactive exploration and translate them into structured specifications, including required UI components, controllable parameters, and expected behavioral responses. These specifications are then standardized into natural language prompts that define interaction requirements without prescribing implementation details. To ensure quality and consistency, all specifications are manually reviewed and refined by multiple annotators, with disagreements resolved through discussion. As shown in Tabel 1, This process yields a corpus of 201 high-quality specifications with varying abstraction levels and interaction complexity. For evaluation, we design an automated Interaction Probe that executes semantically valid user actions on generated applications and detects DOM mutations as a binary signal of responsiveness. Combined with Build Success Rate (BSR), which measures compilability, and Interaction Rate (IR), which measures reactivity, our framework provides a systematic and fine-grained assessment of both execution and interaction behavior.

Using this evaluation framework, we benchmark a diverse set of representative models, including proprietary LLMs, open-source code models, and an API-based agent, on I-WebGenBench. Our results reveal a consistent and substantial gap between compilability and interactivity: while most models achieve high BSR, indicating reliable code generation and deployment, their IR remains significantly lower, demonstrating limited ability to implement functional, event-driven behaviors. Further analysis shows that this gap is primarily driven by deficiencies in interaction logic rather than visual rendering or content alignment. In particular, models often generate structurally complete interfaces with numerous interactive elements, but only a fraction of these respond to user actions. We also identify strong correlations between interaction success and structural properties of the generated code, including state management complexity, layout patterns, and design choices such as visual themes. These findings highlight that achieving true interactivity requires not only syntactic correctness, but also coherent stateful execution, which remains a key limitation of current models.

We summarize our main contributions as follows:

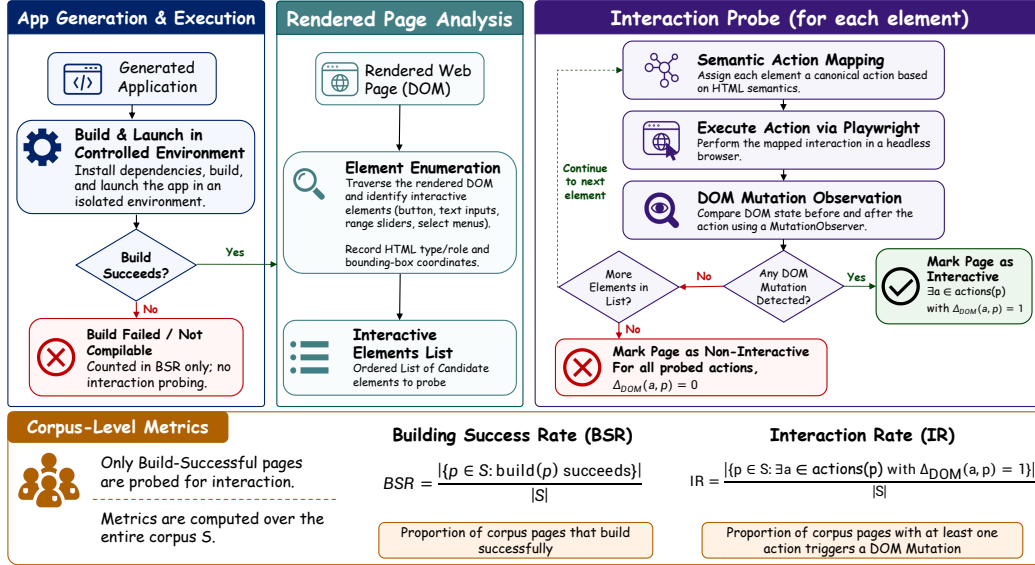


Figure 1: Overview of the I-WebGenBench evaluation pipeline. Generated applications are built in a controlled environment, followed by DOM-based analysis and interaction probing. Interactive elements are enumerated and executed via browser actions, and DOM mutations are used to determine responsiveness. BSR measures compilability, while IR measures whether user actions trigger observable UI changes.

- We introduce I-WebGenBench, the first benchmark for evaluating LLM-generated *interactive scientific web applications* from paper-derived specifications. It comprises 201 curated tasks across five scientific domains with diverse abstraction levels, requiring models to build complete applications from scratch while preserving scientific fidelity and interaction requirements.
- We propose Interaction Probe that executes semantic user actions and detects DOM mutations to measure interactivity. Combined with BSR and IR, our framework explicitly decouples compilability from true user responsiveness, enabling the first systematic and pixel-independent evaluation of interactive behavior.
- We conduct a comprehensive empirical study revealing a persistent compilation–interaction gap in current models. Our analysis shows that interaction failures stem primarily from missing event-driven logic rather than rendering issues, and identifies key structural factors such as state management complexity and layout patterns that govern interactive success.

## 2 Related Work

### 2.1 Benchmarks for Coding LLMs.

Numerous benchmarks have been proposed to evaluate the coding capabilities of large language models. Early evaluations mainly focus on well-specified programming tasks, where models generate functions or short programs that satisfy given test cases [17, 6, 3]. These datasets are typically collected from natural-language programming queries [50], competitive programming tasks [18], automatically synthesized problems [51], and expert-designed evaluations [29]. While effective for measuring code correctness, these benchmarks mainly evaluate function-level generation on relatively small, self-contained programs. More recent benchmarks extend evaluation to repository-level tasks that better reflect real-world development. For example, SWE-bench [21, 46] and SWE-Lancer [28] are built from real software repositories and issue reports, requiring models to implement fixes or features within existing codebases. These tasks involve bug fixing [21, 46, 2], code completion [25, 49], and multi-file patch generation [28].

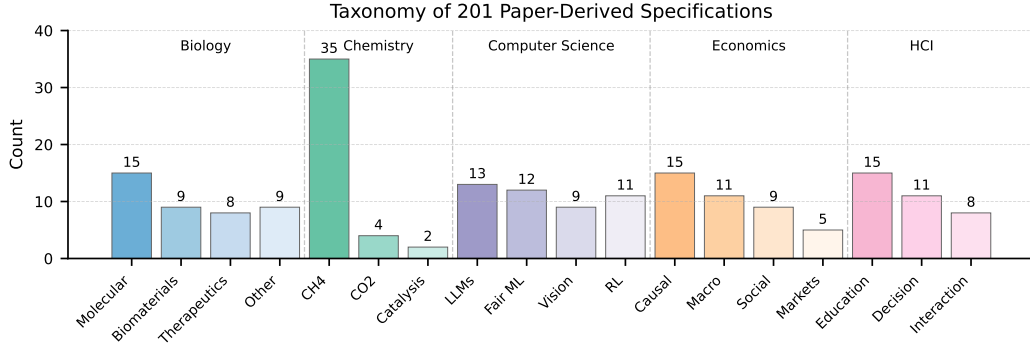


Figure 2: Taxonomy of the 201 paper-derived specifications across five domains. Bars indicate the number of specifications per subcategory. The figure shows the distribution of topics within Biology, Chemistry, Computer Science, Economics, and HCI.

## 2.2 Web Generation Agents

Recent advances in LLMs have enabled systems that assist or automate software and web development. Agent-style frameworks such as OpenHands [35], SWE-agent [45], and Aider [1] allow models to interact with tools and execution environments to iteratively generate and refine code. Development assistants including Cursor [12], GitHub Copilot [11], and Devin [36] further demonstrate the potential of LLM-driven programming systems. Beyond general code generation, recent work explores integrating visual information into coding tasks. In particular, VLM have been studied for generating webpage code from visual inputs, such as reconstructing HTML from webpage screenshots [16, 31, 48, 5]. Other studies evaluate models on implementing interactive elements or solving web development tasks through predefined pipelines [39, 40, 43]. More recent work has begun to explore end-to-end web generation tasks. However, existing settings still primarily focus on static webpage generation. The problem of constructing dynamic web applications, as well as systematically evaluating their interaction behavior, remains underexplored [27, 34, 20, 27, 24, 8]. In particular, there is a lack of benchmarks that assess whether generated interfaces support meaningful, event-driven interactions and how such capabilities vary across models. In contrast, our work introduces a benchmark for dynamic webpage generation, focusing on the ability to construct interactive web systems from scratch and to analyze their interaction behavior.

## 3 I-WebGenBench

### 3.1 Data Curation

We construct I-WebGenBench from scientific paper PDFs spanning five domains: Biology, Chemistry, Computer Science, Economics, and Human-Computer Interaction (HCI). For each paper, the goal is to derive an interaction specification that defines the expected behaviors of an interactive scientific web application, including controllable parameters, dynamic visual components, and user-triggered responses.

To build a high-quality seed set, we recruit three PhD-level annotators for each domain. Papers are sampled from diverse research topics within each field, and annotators independently derive interaction specifications directly from the full PDFs. Each specification contains 3–10 core interaction elements describing parameter manipulation, dynamic updates, and expected system behavior. Across annotators, more than 95% of identified interaction points are consistent, indicating strong agreement on the underlying interactive mechanisms. Disagreements are resolved through discussion to produce unified reference specifications.

To scale beyond fully manual annotation, we design a structured prompting pipeline that takes a paper PDF as input and automatically generates an interaction specification. The prompt guides the model to extract key scientific concepts, dynamic processes, and parameter-dependent behaviors, and to organize them into a standardized description of interactive elements. After calibration on the manually annotated seed set, the generated specifications achieve over 95% overlap with human

references. We therefore apply the same prompting pipeline to the remaining papers to efficiently expand dataset coverage across domains. Each generated specification is expressed as a natural-language prompt describing required UI components, controllable parameters, and expected system behaviors, without prescribing implementation details. Models are required to generate complete interactive applications solely from these specifications.

**Quality Control.** To ensure the reliability of the automatically generated specifications, we conduct an additional expert-review stage across all domains. For each domain, we randomly sample generated specifications and ask three domain experts to independently evaluate them against the source paper PDFs. Experts assess whether a specification accurately captures the essential interactive mechanisms of the paper, including controllable parameters, dynamic behaviors, and expected responses, while also checking for unsupported or spurious content. We adopt a majority-vote filtering protocol: a specification is retained only if at least two of the three experts judge it to be both consistent with the original paper and sufficiently complete for downstream evaluation. Specifications failing to satisfy this criterion are discarded. This additional review process reduces the risk of annotation noise and improves consistency across domains. After filtering, the final dataset contains 201 high-quality specifications spanning five scientific domains, which serve as standardized task definitions for evaluating generated interactive applications.

### 3.2 Evaluation Methods

We evaluate generated applications using two complementary paradigms: rule-based metrics and model-based assessment.

**Rule-Based Metrics.** We first assess execution validity and interaction behavior using deterministic metrics. A generated application is considered valid if it compiles successfully in a controlled environment, yielding the Build Success Rate (BSR):

$$BSR = \frac{|\{p \in \mathcal{S} : \text{build}(p) \text{ succeeds}\}|}{|\mathcal{S}|}. \quad (1)$$

To measure interaction, we define the Interaction Rate (IR) based on whether user actions produce observable effects:

$$IR = \frac{|\{p \in \mathcal{S} : \exists a \in \text{actions}(p) \text{ with } \Delta_{DOM}(a, p) = 1\}|}{|\mathcal{S}|}. \quad (2)$$

Here,  $\Delta_{DOM}(a, p) = 1$  indicates that action  $a$  triggers at least one DOM mutation. To compute IR, we use a deterministic interaction probe based on a headless browser (Playwright). The probe operates on the rendered DOM by identifying interactive elements, applying predefined actions, and detecting resulting DOM changes. Interaction is defined through DOM-level state changes rather than visual differences.

**VLM-as-a-Judge.** To assess properties not captured by rule-based metrics, we employ a VLM as a judge. For each compilable application, the VLM is provided with the interaction specification and a sequence of screenshots collected during interaction probing. Each application is scored on a 100-point scale across multiple dimensions, including semantic alignment, visual quality, interaction fidelity, clarity, and stability. Semantic alignment evaluates consistency with the specification, while visual quality assesses layout and interface design. Interaction fidelity examines whether user actions lead to meaningful updates, and clarity measures interpretability. Stability accounts for runtime errors and rendering failures. This model-based evaluation complements BSR and IR by capturing higher-level semantic and design properties that are not directly observable through programmatic signals. Further implementation details and evaluation protocols are described in the appendix.

## 4 Experiments

### 4.1 Implementation Details

All experiments prompt each large language model  $M$  to generate a complete web application  $A$  from a scientific specification  $S$  defined in a task instance  $T$  from the I-WebGenBench dataset. Each

Table 2: Overall scores, build success rate (BSR), and interaction rate (IR) across five domains. ‘—’ indicates that IR is not reported because the model’s build success rate is insufficient for meaningful interaction evaluation.

| Model                            | Bio  |       |      | Chem |       |      | CS   |       |      | Econ |       |      | HCI  |       |      | AVG  |
|----------------------------------|------|-------|------|------|-------|------|------|-------|------|------|-------|------|------|-------|------|------|
|                                  | All  | BSR   | IR   | All  | BSR   | IR   | All  | BSR   | IR   | All  | BSR   | IR   | All  | BSR   | IR   | All  |
| <b>Based on API-based Models</b> |      |       |      |      |       |      |      |       |      |      |       |      |      |       |      |      |
| GPT-5.4                          | 82.6 | 97.6  | 66.3 | 79.1 | 100.0 | 50.7 | 76.1 | 97.8  | 49.7 | 75.8 | 100.0 | 47.8 | 84.1 | 100.0 | 55.0 | 79.4 |
| Gemini-3.1                       | 61.3 | 90.2  | 47.8 | 50.3 | 78.0  | 41.9 | 45.2 | 82.2  | 27.9 | 52.5 | 87.5  | 29.1 | 62.1 | 97.1  | 39.1 | 53.8 |
| Kimi-K2.5                        | 46.7 | 100.0 | 51.9 | 54.3 | 100.0 | 30.5 | 51.8 | 100.0 | 44.1 | 38.9 | 97.5  | 34.1 | 52.8 | 100.0 | 47.5 | 48.9 |
| Qwen-3.5                         | 55.7 | 100.0 | 13.4 | 58.1 | 100.0 | 15.6 | 41.2 | 93.3  | 19.2 | 43.2 | 100.0 | 15.1 | 38.3 | 94.3  | 12.0 | 47.5 |
| Qwen-3-Max                       | 32.3 | 48.8  | 54.5 | 21.1 | 34.1  | 42.3 | 32.5 | 48.9  | 37.4 | 27.8 | 52.5  | 37.7 | 45.8 | 79.4  | 38.3 | 31.5 |
| MiniMax-M2.7                     | 34.5 | 95.1  | 36.6 | 29.4 | 95.1  | 18.4 | 35.1 | 100.0 | 29.5 | 21.3 | 72.5  | 9.6  | 31.3 | 97.1  | 20.9 | 30.4 |
| DeepSeek-V4-Flash                | 17.5 | 85.4  | 38.3 | 17.2 | 90.2  | 21.6 | 21.6 | 82.2  | 26.6 | 22.3 | 85.0  | 22.1 | 25.7 | 85.3  | 30.6 | 20.7 |
| MiMo-V2.5                        | 22.9 | 95.1  | 44.5 | 12.3 | 41.5  | 35.2 | 17.2 | 66.7  | 28.6 | 14.0 | 52.5  | 34.0 | 15.8 | 67.6  | 54.3 | 16.5 |
| Hunyuan-T1                       | 7.3  | 36.6  | 36.5 | 11.1 | 36.6  | 33.8 | 16.6 | 53.3  | 32.7 | 12.3 | 45.0  | 26.5 | 18.6 | 73.5  | 26.2 | 13.0 |
| DeepSeek-V3.2                    | 9.7  | 12.2  | —    | 9.5  | 12.2  | —    | 8.8  | 13.3  | —    | 11.3 | 20.0  | —    | 5.0  | 0.0   | —    | 9.0  |
| <b>Based on API Agents</b>       |      |       |      |      |       |      |      |       |      |      |       |      |      |       |      |      |
| PaperVoyager (Gemini-3.1)        | 76.3 | 97.6  | 56.3 | 56.6 | 75.6  | 39.5 | 58.8 | 91.1  | 35.5 | 66.0 | 97.5  | 31.7 | 74.1 | 100.0 | 38.3 | 65.9 |
| <b>Based on Open-Source LLM</b>  |      |       |      |      |       |      |      |       |      |      |       |      |      |       |      |      |
| WebGenAgent-SFT-8B               | 5.0  | 0.0   | —    | 5.0  | 0.0   | —    | 5.0  | 0.0   | —    | 5.0  | 0.0   | —    | 5.0  | 0.0   | —    | 5.0  |

generated project is executed in a controlled browser-based environment that installs dependencies, builds the project, and launches it in a Chromium instance. The evaluation system uses Playwright to load the generated interface, execute interaction sequences, and capture runtime browser events. The Interaction Probe loads the application, invokes ExtractUI to identify interactive elements such as buttons, sliders, and input fields from the DOM, and then applies GenerateOps to construct candidate user operations including clicks, parameter adjustments, and text inputs. These operations are executed sequentially through Playwright while a MutationObserver monitors DOM mutation signals to determine whether interface responses occur after each action. For each task instance T, the pipeline records build status and logs interaction traces during automated exploration. All models are evaluated with identical prompts and execution environments, and the same interaction protocol is applied across generated applications to measure behavior under consistent conditions.

**Models.** Unlike prior work that relies on external code-agent frameworks, we directly evaluate the end-to-end capability of VLMs to generate interactive web applications from specifications [27]. All models are prompted to produce complete applications in a single pass without tool use or iterative refinement. The exact prompting protocol is provided in the appendix. We evaluate a diverse set of representative models, including proprietary API-based models, API-based agents, and open-source models. These include GPT-5.4 [32], Gemini-3.1-pro-preview [10], Kimi-K2.5 [33], Qwen-3.5 [44], Qwen-3-Max [44], MiniMax-M2.7 [23], DeepSeek-V4-Flash [15], MiMo-V2.5 [41], Hunyuan-T1, and DeepSeek-V3.2 [15], as well as the API-based agent PaperVoyager [13] and the open-source model WebGenAgent-SFT-8B [27].

## 4.2 Main Results

Table 2 presents a unified evaluation of model performance across five domains by jointly considering overall scores and core capabilities, including BSR and IR. A clear hierarchy emerges: proprietary models consistently dominate across both aggregate performance and underlying capabilities, with GPT-5.4 achieving the best results in all domains, indicating strong end-to-end competence in generating visually coherent, semantically aligned, and functionally executable web applications. However, despite near-saturated BSR values (often above 95%), IR remains substantially lower across all models, revealing a persistent gap between successful code generation and effective interactive behavior implementation. This discrepancy suggests that while most models can reliably produce syntactically correct and compilable interfaces, they struggle to translate user intent into consistent event-driven logic and state updates.

Across domains, performance differences further highlight task-specific challenges. Biology and HCI tasks generally yield higher scores and IR, whereas Computer Science and Economics exhibit consistently lower interaction performance, reflecting the increased difficulty of implementing

Table 3: Detailed Score Distribution Across Models (V: Visual, I: Interaction, T: Topic, C: Clarity, R: Rule). All values are averaged over 201 task instances.

| Model          | V    | I    | T    | C   | R   | Total       | Model             | V    | I    | T    | C   | R   | Total       |
|----------------|------|------|------|-----|-----|-------------|-------------------|------|------|------|-----|-----|-------------|
| GPT-5.4        | 24.5 | 27.8 | 12.9 | 9.1 | 5.0 | <b>79.4</b> | PaperVoyager      | 19.9 | 22.5 | 11.3 | 7.2 | 5.0 | <b>65.9</b> |
| Gemini-3.1-pro | 15.6 | 20.3 | 7.8  | 5.2 | 5.0 | <b>53.8</b> | Kimi-K2.5         | 14.6 | 13.9 | 9.0  | 6.3 | 5.0 | <b>48.9</b> |
| Qwen-3.5       | 21.2 | 16.3 | 5.0  | 0.0 | 5.0 | <b>47.5</b> | Qwen-3-Max        | 7.5  | 10.5 | 5.5  | 3.5 | 4.5 | <b>31.5</b> |
| MiniMax-M2.7   | 9.2  | 6.6  | 5.8  | 3.9 | 5.0 | <b>30.4</b> | DeepSeek-v4-Flash | 5.0  | 6.5  | 3.5  | 2.5 | 3.2 | <b>20.7</b> |
| MiMo-v2.5      | 5.5  | 4.5  | 2.5  | 1.5 | 2.5 | <b>16.5</b> | Hunyuan-T1        | 4.0  | 3.5  | 2.0  | 1.5 | 2.0 | <b>13.0</b> |

Max scores: V(30), I(40), T(15), C(10), R(5).

structured reasoning, algorithmic logic, and precise state transitions in interactive settings. Mid-tier models such as Qwen-3.5 and MiniMax-M2.7 reinforce this pattern: although they achieve high BSR, their low IR indicates a tendency to generate visually complete but functionally shallow interfaces. In contrast, the API-based agent PaperVoyager demonstrates a more balanced profile, achieving competitive scores alongside relatively high IR, which underscores the effectiveness of system-level orchestration in bridging the gap between generation and interaction. At the lower end, models like DeepSeek-V3.2 and WebGenAgent-SFT-8B fail to achieve reliable build success, leading to negligible or unavailable interaction performance. Overall, the unified results expose a fundamental limitation of current approaches: robust code generation does not necessarily translate into meaningful interactivity, and closing this gap remains a central challenge for future web generation systems.

**Gap Between Visual Quality and Interactivity.** Table 3 provides a more fine-grained breakdown of model performance across different evaluation dimensions, including visual quality (V), interaction capability (I), topic alignment (T), clarity (C), and rule compliance (R). Across models, interaction scores exhibit the largest variation and remain substantially lower than visual scores in most cases, suggesting that implementing functional interactivity is consistently more challenging than generating visually plausible interfaces. Proprietary models such as GPT-5.4 achieve relatively balanced performance across all dimensions, while weaker open-source models show pronounced deficiencies in interaction capability and semantic alignment. In particular, several lower-performing models obtain moderate visual scores despite extremely limited interaction performance, indicating a tendency to prioritize static interface generation over executable behavior. Overall, these results further reinforce the gap between surface-level visual quality and meaningful interactive functionality.

**BSR, IR, and Model Behavior.** In the left panel of Figure 3, models exhibit distinct distributions in the two-dimensional space: most models cluster in regions with relatively high BSR (approximately 80%–100%), while IR values are more dispersed, primarily ranging from 10% to 55%. Some models (e.g., GPT-5.4 and PaperVoyager) appear in the upper-right region, characterized by both high BSR and relatively high IR. In contrast, other models (e.g., Qwen-3.5 and MiniMax-M2.7) show high BSR but comparatively low IR, forming a cluster in the lower-right region. Additionally, a small number of models (e.g., Qwen-3-Max) occupy regions with moderate BSR but relatively higher IR, exhibiting patterns that differ from the dominant distribution.

**Effects of Generation Capacity.** The right panel of Figure 3 summarizes model behavior under standardized generation settings. To minimize confounding factors, all models were evaluated using near-deterministic decoding (lowest supported temperature) and maximal output budgets, ensuring that observed differences primarily reflect intrinsic model capabilities rather than sampling variability. Under these conditions, a consistent pattern emerges: while most models can produce compilable code, their success is strongly constrained by generation capacity, including context length and available output budget. Models with restricted capacity exhibit systematically lower build success, indicating that insufficient sequence budget directly limits end-to-end generation performance. At the extreme, models operating under tight constraints fail to produce any valid outputs, highlighting the difficulty of completing complex generation tasks within limited context. These findings suggest that, beyond raw modeling ability, generation capacity—particularly sequence length and available output budget—is a critical bottleneck for real-world usability.

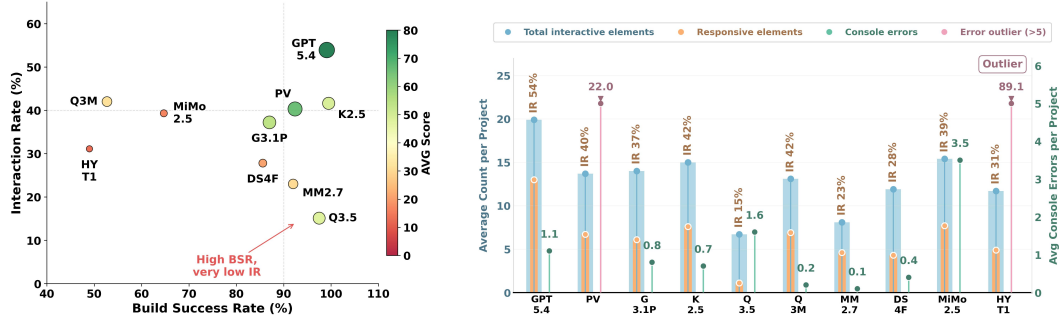


Figure 3: (left) BSR vs IR. Bubble size and color indicate overall score. Most models achieve high BSR but substantially lower IR, confirming that functional interactivity remains the key bottleneck. (right) Interaction probe results: total interactive elements vs. responsive elements (left axis) and average console errors (right axis) per model; IR percentages are shown above each bar. Model abbreviations: GPT 5.4, PV, G3.1P, K2.5, Q3.5, Q3M, MM2.7, DS4F, MiMo 2.5, HYT1.



(a) Kimi-K2.5: Visual 58/70, IR 0%

(b) GPT-5.4: Visual 5/70, IR 70.6%

Figure 4: Visual quality and functional interactivity are orthogonal. (a) Kimi-K2.5 renders a polished interface with slider, scatter plot, and toggle controls, but none respond to user actions. (b) GPT-5.4 produces a visually sparse layout, yet its perturbation grid and variable selectors are fully functional.

## 5 Futher Analysis

Beyond aggregate metrics, we analyze structural properties of the generated code to understand what drives or hinders interactive behavior.

**Visual Quality Does Not Guarantee Interactivity.** Figure 4 compares visual quality with functional interactivity across generated applications. Examples illustrate that visually complete interfaces do not always exhibit corresponding interactive behavior. In some cases, interfaces contain multiple rendered controls but show no measurable interaction (IR=0%), while other examples with relatively minimal layouts achieve substantially higher IR. This variation indicates that visual completeness and interaction behavior can differ across generated outputs.

**Moderate State Complexity Tends to Work Best.** We count reactive state declarations per application as a proxy for state complexity. Table 4(a) shows a non-monotonic relationship. Very few state variables (0–2) yield low scores because the application lacks reactive bindings for meaningful interaction. Scores rise sharply at moderate levels (3–8), where the best average scores appear. Beyond that, scores gradually drop: applications with more than 20 state variables score notably lower. Most samples fall in the 3–17 range; extremes are rare.

**Sidebar Layouts Are Associated with Higher Interaction.** Only 4.5% of evaluated applications (63/1,410) include a sidebar. Table 4(b) shows that these applications consistently perform better: higher mean score (36.9 vs. 33.9), more click handlers (9.6 vs. 7.0), and higher interaction rate (51.1% vs. 38.1%). They also more often implement explicit view-switching logic (59% vs. 24%), where a dedicated state variable controls the active panel and each navigation item triggers a transition.



Table 4: Structural analysis of generated applications ( $n = 1,410$ ). (a) Average score by number of reactive state variables. (b) Performance comparison between sidebar and non-sidebar layouts.

| State variables      | $n$ | Avg score |                     | Sidebar<br>( $n = 63$ ) | Non-sidebar<br>( $n = 1,347$ ) |
|----------------------|-----|-----------|---------------------|-------------------------|--------------------------------|
| 0–2                  | 33  | 8.9       |                     |                         |                                |
| 3–8                  | 551 | 35.2      |                     |                         |                                |
| 9–17                 | 795 | 34.4      |                     |                         |                                |
| 18–20                | 24  | 31.5      |                     |                         |                                |
| 21+                  | 7   | 25.9      |                     |                         |                                |
| (a) State complexity |     |           |                     | (b) Layout comparison   |                                |
|                      |     |           | Mean score          | 36.9                    | 33.9                           |
|                      |     |           | Avg click handlers  | 9.6                     | 7.0                            |
|                      |     |           | Interaction rate    | 51.1%                   | 38.1%                          |
|                      |     |           | Uses view-switching | 59%                     | 24%                            |

Despite their small share, sidebar layouts are systematically associated with stronger interaction metrics across the dataset. We cannot fully rule out selection effects, but the pattern holds across models of different capabilities, suggesting that sidebar architecture itself helps implementation quality.

**Visual Habit vs. Functional Performance.** Beyond structural components, we examine whether a model’s visual choices correlate with its functional success. Table 5 summarizes the distribution of theme preferences and their corresponding performance impacts (scaled to 30 points). We observe a clear **Light-theme Dominance** across the corpus, most notably in GPT-5.4, which defaults to light interfaces in 97% of cases, with a dark-mode ratio of only 3.0%. However, this stylistic preference appears to reflect inherited design conventions rather than deliberate functional optimization. While light-themed applications sometimes achieve slightly higher average scores—for instance, GPT-5.4 averages 23.8 for light themes compared with 22.4 for dark themes—the differences remain relatively small across models. This suggests that while LLMs inherit a visual preference for white-background scientific dashboards from their training data, their ability to implement complex interaction logic remains largely independent of surface-level UI theme choices.

Table 5: Theme distribution and corresponding average scores (scaled to 30 points).

| Model             | Theme Distribution (Count) |      |        | Avg. Score by Theme (out of 30) |      |        |
|-------------------|----------------------------|------|--------|---------------------------------|------|--------|
|                   | Light                      | Dark | Others | Light                           | Dark | Others |
| GPT-5.4           | 192                        | 6    | 1      | 23.8                            | 22.4 | 23.0   |
| Gemini-3.1-pro    | 111                        | 27   | 35     | 16.4                            | 15.1 | 15.9   |
| Kimi-K2.5         | 144                        | 32   | 24     | 15.0                            | 13.7 | 14.5   |
| PaperVoyager      | 108                        | 39   | 37     | 20.3                            | 18.6 | 19.6   |
| MiniMax-M2.7      | 107                        | 63   | 13     | 9.6                             | 8.3  | 9.1    |
| DeepSeek-v4-Flash | 128                        | 40   | 4      | 6.5                             | 5.3  | 5.9    |

## 6 Conclusion

We introduced I-WebGenBench, the first benchmark for evaluating whether LLMs can generate interactive scientific web applications from paper-derived specifications. Across 201 tasks spanning five scientific domains, our benchmark combines deterministic interaction probing with multi-dimensional evaluation to explicitly distinguish compilability from genuine event-driven responsiveness. Experiments across diverse models reveal a persistent compilation–interaction gap: although modern LLMs can reliably generate visually complete and compilable interfaces, they still struggle to implement coherent interactive behavior. Our findings establish I-WebGenBench as a new testbed for advancing interactive and stateful web generation.

**Limitations.** Although I-WebGenBench spans five scientific domains, it still covers only a subset of possible interactive scientific applications. More complex settings, such as collaborative multi-user systems, long-running simulations, and highly specialized scientific workflows, are not included in the current benchmark. In addition, while the VLM-as-a-judge protocol complements deterministic metrics, qualitative assessments of visual quality and educational value may still introduce subjective bias. Finally, due to budget and API access constraints, we were unable to include certain frontier proprietary models such as Claude in the current evaluation.

## References

- [1] Aider-AI. Ai pair programming in your terminal, 2024. URL <https://github.com/Aider-AI/aider>. Accessed: 2025-04-22.
- [2] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. Swe-bench+: Enhanced coding benchmark for llms. *arXiv preprint arXiv:2410.06992*, 2024.
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [4] Emad Barsoum and Falko Kuester. Webvr: an interactive web browser for virtual environments. In *Stereoscopic Displays and Virtual Reality Systems XII*, volume 5664, pages 540–547. Spie, 2005.
- [5] Tony Beltramelli. pix2code: Generating code from a graphical user interface screenshot, 2017. URL <https://arxiv.org/abs/1705.07962>.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [7] Qi Chen, Dileepa Pitawela, Chongyang Zhao, Gengze Zhou, Hsiang-Ting Chen, and Qi Wu. Webvln: Vision-and-language navigation on websites, 2023.
- [8] Yang Chen, Minghao Liu, Yufan Shen, Yunwen Li, Tianyuan Huang, Xinyu Fang, Tianyu Zheng, Wenxuan Huang, Cheng Yang, Daocheng Fu, et al. Iwr-bench: Can llms reconstruct interactive webpage from a user interaction video? *arXiv preprint arXiv:2509.24709*, 2025.
- [9] Yuhang Chen, Tianpeng Lv, Siyi Zhang, Yixiang Yin, Yao Wan, Philip S Yu, and Dongping Chen. Paper2web: Let’s make your paper alive! *arXiv preprint arXiv:2510.15842*, 2025.
- [10] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- [11] GitHub Copilot. Github copilot, 2024. URL <https://github.com/features/copilot>. Accessed: 2025-04-22.
- [12] Cursor. Cursor: The ai code editor, 2024. URL <https://www.cursor.com/>. Accessed: 2025-04-22.
- [13] Dasen Dai, Biao Wu, Meng Fang, and Wenhao Wang. Papervoyager: Building interactive web with visual language models. *arXiv preprint arXiv:2603.22999*, 2026.
- [14] Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Agentic reinforced policy optimization, 2025. URL <https://arxiv.org/abs/2507.19849>.
- [15] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [16] Hongcheng Guo, Wei Zhang, Junhao Chen, Yaonan Gu, Jian Yang, Junjia Du, Binyuan Hui, Tianyu Liu, Jianxin Ma, Chang Zhou, and Zhoujun Li. Iw-bench: Evaluating large multimodal models for converting image-to-web, 2024. URL <https://arxiv.org/abs/2409.18980>.
- [17] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.
- [18] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [19] Juyong Jiang, Chansung Park, Jiasi Shen, Sunghun Kim, Jianguo Li, Yue Wang, et al. Webgen-r1: Incentivizing llms to generate functional and aesthetic websites with reinforcement learning.
- [20] Juyong Jiang, Chansung Park, Jiasi Shen, Sunghun Kim, Jianguo Li, Yue Wang, et al. Webgen-r1: Incentivizing llms to generate functional and aesthetic websites with reinforcement learning. 2026.

- [21] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- [22] Xinping Lei, Xinyu Che, Junqi Xiong, Chenchen Zhang, Yukai Huang, Chenyu Zhou, Haoyang Huang, Minghao Liu, Letian Zhu, Hongyi Ye, et al. Webcompass: Towards multimodal web coding evaluation for code language models. *arXiv preprint arXiv:2604.18224*, 2026.
- [23] Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025.
- [24] Chenxu Liu, Yingjie Fu, Wei Yang, Ying Zhang, and Tao Xie. Webcoderbench: Benchmarking web application generation with comprehensive and interpretable evaluation metrics. *arXiv preprint arXiv:2601.02430*, 2026.
- [25] Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023.
- [26] Yuxuan Lu, Bingsheng Yao, Hansu Gu, Jing Huang, Jessie Wang, Laurence Li, Jiri Gesi, Qi He, Toby Jia-Jun Li, and Dakuo Wang. Uxagent: An llm agent-based usability testing framework for web design. *arXiv preprint arXiv:2502.12561*, 2025.
- [27] Zimu Lu, Yunqiao Yang, Houxing Ren, Haotian Hou, Han Xiao, Ke Wang, Weikang Shi, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. Webgen-bench: Evaluating llms on generating interactive and functional websites from scratch. *arXiv preprint arXiv:2505.03733*, 2025.
- [28] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. Swe-lancer: Can frontier llms earn \$1 million from real-world freelance software engineering? *arXiv preprint arXiv:2502.12115*, 2025.
- [29] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro Von Werra, and Shayne Longpre. Octopack: Instruction tuning code large language models. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, 2023.
- [30] Jingwei Shi, Zeyu Zhang, Biao Wu, Yanjie Liang, Meng Fang, Ling Chen, and Yang Zhao. Presentagent: Multimodal agent for presentation video generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 760–773, 2025.
- [31] Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. Design2code: Benchmarking multimodal code generation for automated front-end engineering, 2025. URL <https://arxiv.org/abs/2403.03163>.
- [32] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- [33] Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chenzhuang Du, Chu Wei, et al. Kimi-vl technical report. *arXiv preprint arXiv:2504.07491*, 2025.
- [34] Kuang-Da Wang, Zhao Wang, Yotaro Shimose, Wei-Yao Wang, and Shingo Takamatsu. Webgen-v bench: Structured representation for enhancing visual design in llm-based web generation and evaluation. *arXiv preprint arXiv:2510.15306*, 2025.
- [35] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2024.
- [36] Scott Wu. Introducing devin, the first ai software engineer, 2024. URL <https://cognition.ai/blog/introducing-devin>. Accessed: 2025-04-22.
- [37] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [38] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang, Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang, and Michael R Lyu. Interaction2code: Benchmarking mllm-based interactive webpage code generation from interactive prototyping. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 241–253. IEEE, 2025.

- [39] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang, Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang, and Michael R. Lyu. Interaction2code: Benchmarking mllm-based interactive webpage code generation from interactive prototyping, 2025. URL <https://arxiv.org/abs/2411.03292>.
- [40] Jingyu Xiao, Ming Wang, Man Ho Lam, Yuxuan Wan, Junliang Liu, Yintong Huo, and Michael R. Lyu. Designbench: A comprehensive benchmark for mllm-based front-end code generation, 2025. URL <https://arxiv.org/abs/2506.06251>.
- [41] LLM Xiaomi, Bingquan Xia, Bowen Shen, Dawei Zhu, Di Zhang, Gang Wang, Hailin Zhang, Huaqiu Liu, Jiebao Xiao, Jinhao Dong, et al. Mimo: Unlocking the reasoning potential of language model—from pretraining to posttraining. *arXiv preprint arXiv:2505.07608*, 2025.
- [42] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. Swe-fixer: Training open-source llms for effective and efficient github issue resolution, 2025. URL <https://arxiv.org/abs/2501.05040>.
- [43] Kai Xu, YiWei Mao, XinYi Guan, and ZiLong Feng. Web-bench: A llm code benchmark based on web standards and frameworks, 2025. URL <https://arxiv.org/abs/2505.07473>.
- [44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [45] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [46] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024.
- [47] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2022.
- [48] Sukmin Yun, Haokun Lin, Rusiru Thushara, Mohammad Qazim Bhat, Yongxin Wang, Zutao Jiang, Mingkai Deng, Jinhong Wang, Tianhua Tao, Junbo Li, Haonan Li, Preslav Nakov, Timothy Baldwin, Zhengzhong Liu, Eric P. Xing, Xiaodan Liang, and Zhiqiang Shen. Web2code: A large-scale webpage-to-code dataset and evaluation framework for multimodal llms, 2024. URL <https://arxiv.org/abs/2406.20098>.
- [49] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023.
- [50] Shudan Zhang, Hanlin Zhao, Xiao Liu, Qinkai Zheng, Zehan Qi, Xiaotao Gu, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. Naturalcodebench: Examining coding performance mismatch on humaneval and natural user prompts. *arXiv preprint arXiv:2405.04520*, 2024.
- [51] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

## A Implementation Details

In our experiments, for each model we consistently used the **maximum allowed completion tokens** to ensure sufficient output capacity. For the temperature parameter, we set it to the **lowest supported value** to favor deterministic (greedy) decoding. Specifically, most models support `temperature = 0.0`. For MiniMax-M2.7 and MiMo-v2.5, the valid range is  $(0, 1]$ , so we used the smallest safe value 0.01. GPT-5.4 does not support temperature at all and is marked as **Not Supported**. All other settings follow the same principle of minimizing temperature while respecting each API’s constraints.

Table 6: Maximum output tokens and minimum usable temperature per model.

| Model              | Max Output Tokens | Temperature   |
|--------------------|-------------------|---------------|
| GPT-5.4            | 128,000           | Not Supported |
| Gemini-3.1-pro     | 65,536            | 0.0           |
| Kimi-K2.5          | 262,144           | 1.0           |
| Qwen-3.5           | 16,384            | 0.6           |
| Qwen-3-Max         | 65,536            | 0.0           |
| MiniMax-M2.7       | 196,600           | 0.01          |
| DeepSeek-V3.2      | 64,000            | 0.0           |
| Hunyuan-T1         | 64,000            | 0.0           |
| MiMo-v2.5          | 131,072           | 0.01          |
| DeepSeek-v4-Flash  | 384,000           | 0.0           |
| WebGenAgent-SFT-8B | 8,192             | 0.0           |

## B Additional Experimental Results

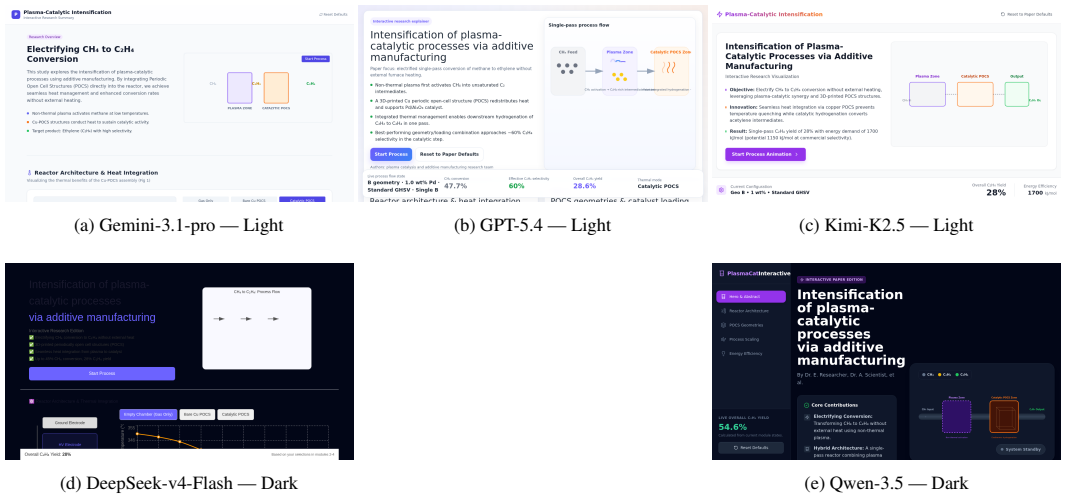


Figure 5: Visual variety of scientific web applications generated from the same specification (plasma-catalytic intensification). While most proprietary models (a–c) adhere to a conventional light-themed academic aesthetic, DeepSeek-v4-Flash and Qwen-3.5 (d–e) default to dark-slate dashboards. Notably, (e) exhibits a complex sidebar structure but hallucinates non-existent author names, illustrating the trade-off between structural complexity and semantic fidelity.

### B.1 Evaluation Methods

Our evaluation framework is designed to capture both the objective compilability of the generated artifacts and their subjective alignment with complex scientific requirements. To achieve this, we combine a deterministic programmatic probe with an expert-level Vision-Language Model (VLM) judge, resulting in a comprehensive 100-point scoring system alongside foundational build metrics.

**Interaction Probe.** To distinguish between pages that merely render and those that exhibit meaningful responses, we introduce a deterministic *Interaction Probe*. Implemented using Playwright as the headless automation backend, the probe proceeds in three stages:

1. *Element Enumeration*: We traverse the fully rendered page to identify interactive elements (e.g., buttons, range sliders, select menus), annotating them with bounding-box coordinates for robust targeting.
2. *Semantic Action Mapping*: Elements are assigned canonical interactions based on their HTML semantics (e.g., populating text inputs, setting sliders to midpoints) to prevent arbitrary failure modes.
3. *DOM Mutation Observation*: For each action, we capture the DOM state before and after execution using a `MutationObserver` configured to track child lists, subtrees, and attributes.

Here,  $\Delta_{DOM}(a, p) = 1$  if action  $a$  triggers at least one DOM mutation. While BSR and IR provide binary indicators of structural viability, assessing the scientific fidelity and educational quality of these interactions necessitates higher-level semantic reasoning.

**VLM-Driven 100-Point Scoring Protocol.** To evaluate the qualitative dimensions of the generated applications, we deploy a state-of-the-art VLM acting as an expert Academic Peer Reviewer and Frontend Architect. For each compilable artifact, the VLM is provided with the human-curated interaction specification alongside a chronological sequence of screenshots captured during the Interaction Probe. The application is strictly evaluated on a 100-point scale distributed across five dimensions:

- **Topic & Semantic Alignment (15 pts)**: Verifies adherence to the *Data Curation* guidelines. The VLM compares the generated application against the original paper specification, penalizing hallucinated concepts, missing interactive modules, or omitted adjustable parameters. A high score indicates high fidelity to the core interactive points annotated by human experts.
- **Visual Aesthetics (30 pts)**: Evaluates UI/UX design quality. Applications are judged on typography, color harmony, and layout maturity, distinguishing between unstyled HTML prototypes and production-ready scientific dashboards.
- **Interaction Fidelity (40 pts)**: While IR proves a state change occurred, this dimension evaluates whether the change is *scientifically meaningful*. By analyzing pre- and post-action screenshots, the VLM assesses if UI manipulations (e.g., adjusting a parameter) result in logically sound updates to charts, simulations, or mathematical models.
- **Clarity & Educational Value (10 pts)**: Assesses the intuitiveness of the visualization. High scores require clear axis labels, self-explanatory controls, and guided interaction flows, ensuring the underlying scientific concept is broadly accessible.
- **Rule & Stability (5 pts)**: Calculated deterministically by the automated probe. Applications start with 5 points, incurring penalties for fatal rendering errors (e.g., entirely blank screens) or critical JavaScript exceptions captured during the interaction cycle.

This hybrid approach ensures the benchmark is grounded in verifiable execution (BSR/IR) while leveraging VLMs to grade the nuanced, multi-modal reality of scientific web application design.

## C Prompt Templates

This section details the prompt templates used across the I-WebGenBench pipeline, covering the entire lifecycle from initial document analysis to the final visual audit. We provide the specific system instructions and user templates for task specification generation (§C.1), monolithic code generation (§C.2), iterative self-repair (§C.3), block-level decomposition and assembly (§C.4), and VLM-based visual evaluation (§C.5). All models receive identical prompts to ensure a fair comparison of their reasoning and generation capabilities; per-model hyper-parameters are listed in Table 6.

### C.1 Task Specification Generation Prompt

The first stage of the pipeline involves converting a raw scientific PDF into a structured, interactive web application specification. This “Task Definition” serves as the ground truth for subsequent code

generation. The prompt (Figure 6) directs the AI to act as an expert researcher and educational UX designer, ensuring that the resulting design is both scientifically accurate and pedagogically effective. This step is crucial for bridging the gap between dense academic text and interactive visual logic.

**Task Specification Generation — System Instruction**

You are an expert AI Researcher and Educational UX Designer specializing in Human-Computer Interaction (HCI) and Scientific Communication. Your task is to conduct a deep reading of the provided academic paper (PDF) and design an immersive, interactive, educational Web Application (React + TypeScript) that serves as a living visualization of the paper’s core contributions.

**Core Mandates:**

- **Accuracy:** Deeply interpret the mathematical mechanisms, experimental workflows, and causal relationships described in the paper.
- **Interactivity:** Design 3–10 “Core Interactive Points” (adjustable parameters, dynamic visual elements, and real-time feedback loops) that allow users to explore “what-if” scenarios.
- **Structure:** Organize the application into 5 distinct modules: 1) Hero/Abstract (visual hook), 2) Architecture/Methodology (interactive diagram), 3) Core Simulation/Experiment (primary playground), 4) Results/Analysis (interactive charts), and 5) Conclusion (synthesis).

**Technical Constraints:**

- Output a structured natural language specification focusing on UI components, state variables, and visual logic.
- Ensure the specification is entirely in the requested output language.

Figure 6: System prompt for the PDF-to-Specification stage. This identifies the core scientific contributions and maps them to interactive UI components, providing a structured blueprint for the generation agents.

## C.2 Code Generation Prompt

Each benchmark instance consists of a *system prompt* (Figure 7) and a *per-paper user prompt* drawn from the specification corpus (`prompts/{domain}_flat/*.txt`,  $\approx 40$  lines per instance). The user prompt encodes the paper title, five interactive module descriptions, visual-style requirements, and technical-stack constraints. The system prompt constrains the output format and establishes the quality bar for interactivity.

**Code Generation — System Prompt**

You are an expert React/TypeScript developer specializing in interactive scientific visualizations.

**Output format.** Return **only** the TSX source for `src/App.tsx`. Do not include markdown fences, explanatory text, or file-path headers. The file must contain a valid default export (`export default function App() {...}`).

**Content requirements.**

- Build an *interactive educational research interface*, not a static landing page or navigation shell.
- The first viewport must render meaningful scientific content together with clear interactive affordances (tabs, sliders, toggles, parameter controls, animated diagrams, or data-linked charts).
- Implement all five modules described in the specification with functional state management and user-driven data exploration.

**Language.** Use English for all user-facing UI text unless the specification explicitly requests another language.

Figure 7: System prompt prepended to every model call during code generation. The identical prompt is used across all baseline models and for the PaperVoyager block-level generation strategy.

### C.3 Iterative Self-Repair Prompt

When a generated application fails to build, the pipeline invokes the same model with the repair prompt shown in Figure 8. Up to  $K=5$  repair rounds are attempted; each round provides the full build-error log (truncated to 3 000 characters) and the current source file (up to 40 000 characters). If the application still does not compile after five rounds, it receives a score of 0 across all evaluation dimensions.

**Iterative Self-Repair — Round  $k/5$**

The following React/TypeScript application failed to compile. Return **only** the complete, corrected `App.tsx`. Do not include markdown fences, explanations, or commentary.

**Build error** (last 3 000 characters):  
{build\_error}

**Current source** (`App.tsx`, up to 40 000 characters):  
{source\_code}

Return the **complete** fixed file. Do not omit unchanged sections.

Figure 8: Repair prompt template. Placeholders `{build_error}` and `{source_code}` are filled at runtime. The round counter  $k$  is shown for illustration; all five rounds use the same template.

### C.4 Block Pipeline Prompts

To handle the complexity of full-scale scientific applications, we introduce a “Block Pipeline” that decomposes the task into manageable UI units. This process uses two primary prompts: a *Splitter* to break down the specification (Figure 9) and a *Merger* to synthesize the resulting code (Figure 10). This modular approach allows for more granular control over component quality and state management.

**Block Pipeline — Splitter Prompt**

Analyze the following Scientific Web App Specification and decompose it into a sequence of independent, functional React blocks. Each block must be a logical UI section (e.g., Sidebar, Parameter Panel, Main Simulation Canvas, Data Table).

**Output Format:** Return a JSON array of Block Objects containing:

- **name:** Unique identifier for the block.
- **description:** Detailed visual and functional instructions for this specific block.
- **state\_dependencies:** List of shared state variables this block must read or write.
- **required\_imports:** Any specialized libraries (e.g., `recharts`, `framer-motion`).

Figure 9: The Splitter prompt used to decompose a monolithic specification into discrete, implementable UI blocks.

### C.5 Visual Evaluation Prompt

The evaluation stage utilizes a state-of-the-art VLM acting as a Senior Principal Frontend Engineer and an Academic Peer Reviewer. The evaluator performs a high-fidelity visual and semantic audit of the generated application. The prompt (Figure 11) directs the model to score the artifact across four qualitative dimensions. This assessment is combined with automated build and interaction probes to produce a final 100-point score.



Block Pipeline — Merger Prompt

You are a Lead Software Architect. I will provide you with several code blocks, each representing a part of a React application. Your task is to assemble them into a single, high-quality `App.tsx` file.

**Instructions:**

- Use a cohesive layout (e.g., a Sidebar + Main Content area).
- Ensure all `useState` and `useEffect` hooks are correctly integrated at the top level or within respective components.
- Resolve any variable naming conflicts and ensure the final app exports a single `default function App()`.
- The final code must be production-ready and compile without errors.

Figure 10: The Merger prompt used to recombine individual blocks into a cohesive, bug-free `App.tsx`.

VLM Evaluation Judge — System Instruction

You are a Senior Principal Frontend Engineer and an Academic Peer Reviewer. You are tasked with evaluating a generated Scientific Web Application based on its visual excellence, interaction fidelity, and semantic alignment with a provided research specification.

You are judging the application on a 100-point scale, distributed across four high-level dimensions (the 5th dimension “Rule” is handled by automated scripts).

**Evaluation Rubrics:**

- **Visual Aesthetics (Max 30 pts):**
  - 25–30: Outstanding. Effective use of whitespace, typography, and color theory.
  - 15–24: Average. Functional but lacks professional polish.
  - 0–14: Poor. Broken layouts, unstyled elements, or jarring colors.
- **Interaction Fidelity & Responsiveness (Max 40 pts):**
  - 30–40: Dynamic. Interactions trigger meaningful, scientifically accurate state changes.
  - 15–29: Basic. Response is delayed, generic, or only partially functional.
  - 0–14: Static. No visible change despite user input.
- **Topic & Semantic Alignment (Max 15 pts):**
  - 12–15: Perfect Alignment. Covers >95% of requested interactive points.
  - 7–11: Partial Alignment. Missing key modules or parameters.
  - 0–6: Divergent. Hallucinated features or significant omissions.
- **Clarity & Educational Value (Max 10 pts):**
  - 8–10: Intuitive. Concepts are clear through exploration and labeling.
  - 0–7: Confusing. Lack of labels or cryptic user interface.

**Output Format:** Return ONLY a JSON object containing the scores and reasoning for each of the four dimensions.

Figure 11: The VLM Evaluation prompt template. This multidimensional rubric ensures that generated applications are judged not only on their code quality but also on their scientific and educational utility.

## NeurIPS Paper Checklist

### 1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction clearly describe the main contributions and scope of the paper, which are supported by the experimental results and analysis presented in later sections.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

## 2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper discusses key limitations, including assumptions, scope of evaluation, and potential constraints, in a dedicated limitations section.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

## 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results requiring formal proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.

- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

#### 4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper provides detailed descriptions of datasets, model configurations, training procedures, and evaluation protocols necessary for reproducing the results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

#### 5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The code and data will be released upon acceptance of the paper, and sufficient experimental details are provided to ensure reproducibility.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.

- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

## 6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper specifies datasets, baselines, hyperparameters, and training configurations required to understand the experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

## 7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The experiments are computationally expensive, making multiple runs impractical; therefore, we report single-run results, while observing consistent trends across different settings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

## 8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper includes information about the hardware setup and approximate computational requirements for the experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

## 9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research complies with the NeurIPS Code of Ethics and does not raise ethical concerns beyond those discussed.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

## 10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses potential benefits as well as possible risks and unintended consequences of the proposed approach.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate

to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

## 11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not introduce assets with significant risk of misuse that would require additional safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

## 12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All external datasets and codebases used in the paper are properly cited and their usage complies with their respective licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, [paperswithcode.com/datasets](https://paperswithcode.com/datasets) has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

### 13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce new datasets, models, or other assets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

### 14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

### 15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects and therefore does not require IRB approval.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

### 16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

Answer: [N/A]

Justification: LLMs are not used as part of the core methodology of this work.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.